

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 3/30

Hello C++, Part 2: I/O Streams _{cont.}, Variables, Objects, and Fundamental types

- 2nd program: Say a greeting to a specific person
- Variables and Objects
- Fundamental types: integrals, floating-points, characters, and Booleans
- The sizeof operator
- The C++14 Digit separators
- References and further readings
- Q&A

120 min (incl. Q & A)



E_{xtending Hello, world!:} Hello to a specific person

E

xtending Hello, world!: Hello to a specific person

- Once we can write text, the logical next step is to read it.

E

xtending Hello, world!: Hello to a specific person

- Once we can write text, the logical next step is to read it.

```
#include <iostream>
#include <string>
int main()
{
    // ask for the person's name
    std::cout << "Please enter your first name: ";
    // read the name
    std::string name;           // define name
    std::cin >> name;           // read into name
    // write a greeting
    std::cout << "Hello, " << name << "!" << '\n';

    return 0;
}
```

E

xtending Hello, world!: Hello to a specific person

- Once we can write text, the logical next step is to read it.

```
#include <iostream>
#include <string>
int main()
{
    // ask for the person's name
    std::cout << "Please enter your first name: ";
    // read the name
    std::string name;          // define name
    std::cin >> name;          // read into name
    // write a greeting
    std::cout << "Hello, " << name << "!" << '\n';

    return 0;
}
```

```
$ g++ greeting.cpp -o greeting
```

```
$ ./greeting
```

```
Please enter your first name: Saeed
```

```
Hello, Saeed!
```

Extending Hello, world!: Hello to a specific person

- Once we can write text, the logical next step is to read it.

```
#include <iostream>
#include <string>
int main()
{
    // ask for the person's name
    std::cout << "Please enter your first name: ";
    // read the name
    std::string name;           // define name
    std::cin >> name;           // read into name
    // write a greeting
    std::cout << "Hello, " << name << "!" << '\n';

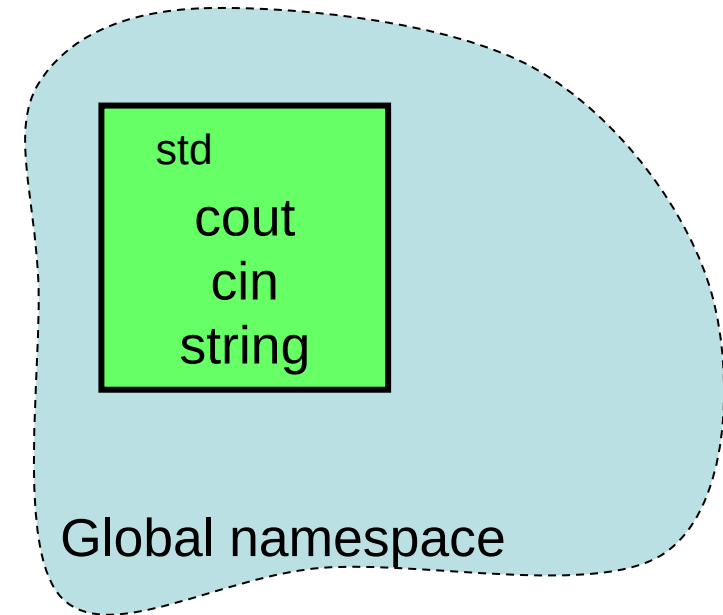
    return 0;
}
```

```
$ g++ greeting.cpp -o greeting
```

```
$ ./greeting
```

```
Please enter your first name: Saeed
```

```
Hello, Saeed!
```



The anatomy of greeting program

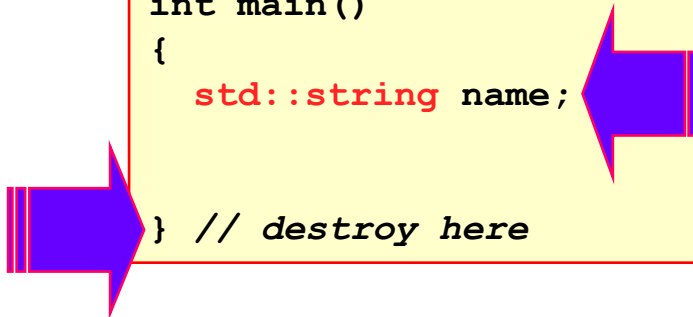
```
std::cout << "Please enter your first name: "; // wait at the current line
```

```
std::string name; // a definition
```

- Because **name** is defined in a function (main) it is **local variable**.

```
int main()
{
    std::string name; // local variable
}
```

```
int main()
{
    std::string name; // create here
} // destroy here
```



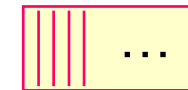
```
int i; // int is a built-in type
```

```
std::string name; // string is a user-define type
```

- Initialization

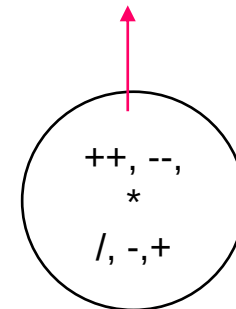
```
std::string name; // null or empty string
```

int type:



Structure

Behavior

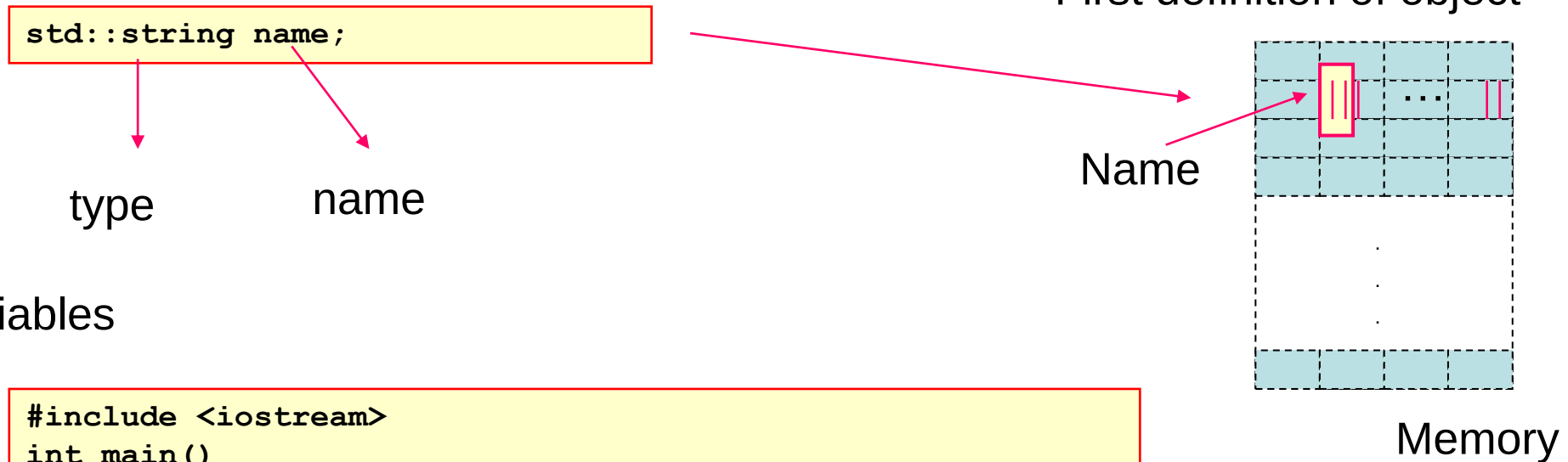


Variable and Object

- Variable

- A variable is a **container**.
- A variable is an **object** that has a name. An object, in turn, is a part of the computer's memory with a type.

Variable : name + type

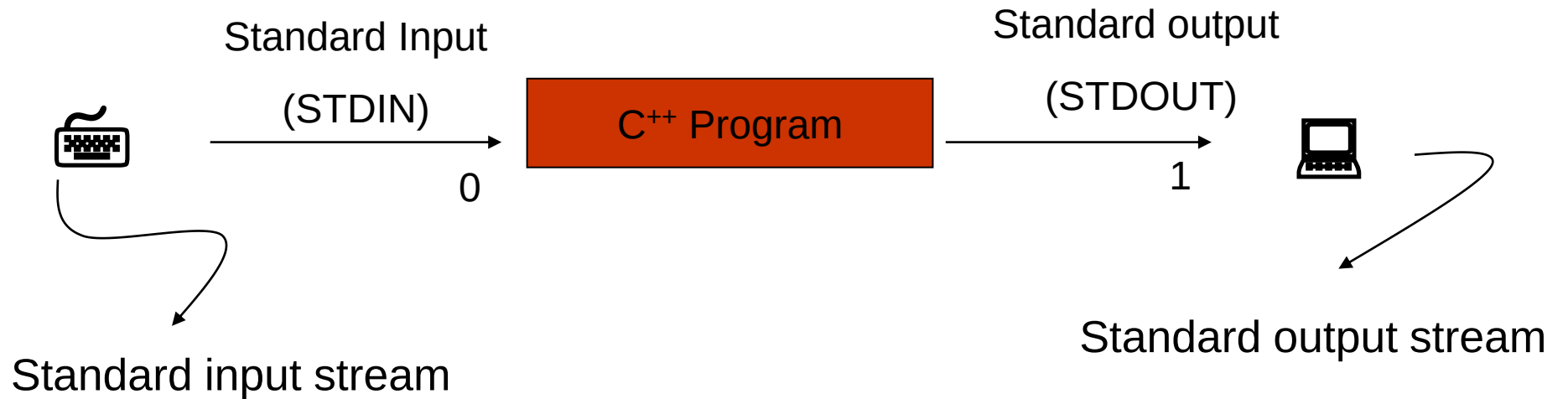


- Symbolic variables

[illegible]

Standard library and input

```
std::cin >> name; // read into name
```



- Input operation:
 - Input operator: `>>`
 - get from
 - Extraction operator (more technical sounding name)
 - cin, an istream class object

Input operator

- >> operator skips **whitespace**.
- **Whitespace characters**: space, tab, backspace, or the end of the line.

```
std::string name;  
std::cin >> name; // read a word: Saeed
```

-----Saeed-----



```
int i;  
std::cin >> i; // read an int:12345
```

-----12345.6-----



I dentifiers and Keywords

- C/C++ is *case-sensitive* language. Uppercase and lowercase letters are distinct.
 - Count, count, COUNT, CounT: four different C++ identifiers.
- C++ **Keywords**:
 - up until now: **int**, **return**
 - bool, class, template, try, ...
- C++ **Reserved words**: include, main, string, vector

T

ypes, identifiers, and names

```
x = y + f(2);
```

x,
y,
f

Names

=
+
()

Operators

- Every name (identifier) has a type associated with it.

```
float x; // x is a floating point variable  
int y = 7; // y is an integer variable with the initial value 7  
float f(int); // f is a function taking an argument of type int and returning a  
              // floating point number
```

- Name = Identifier
- An **identifier** is a sequence of letters and digits. The first character should be letter.

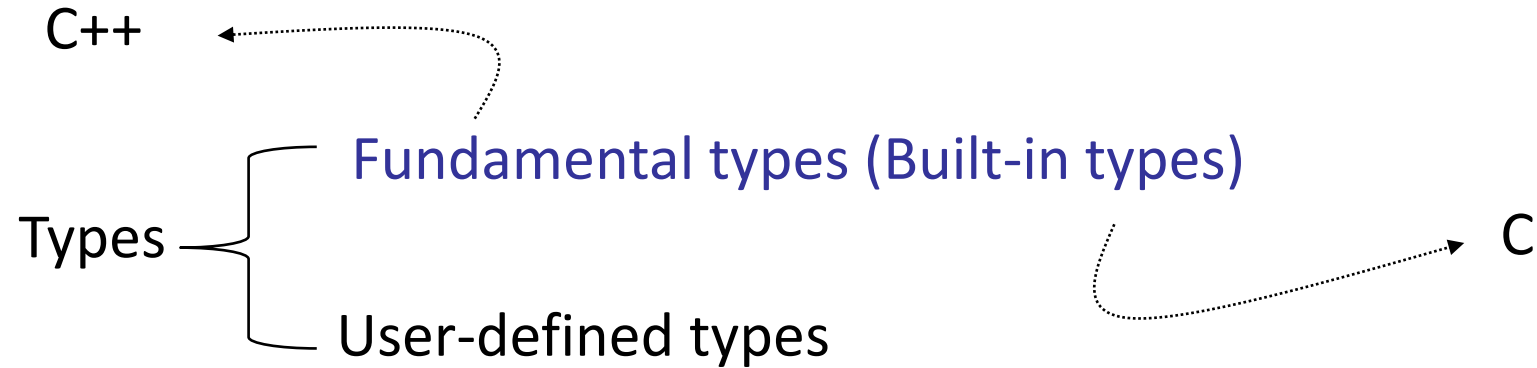
Letter: A-Z, a-z, _

Digit: 0-9

Fundamental types?



Fundamental types



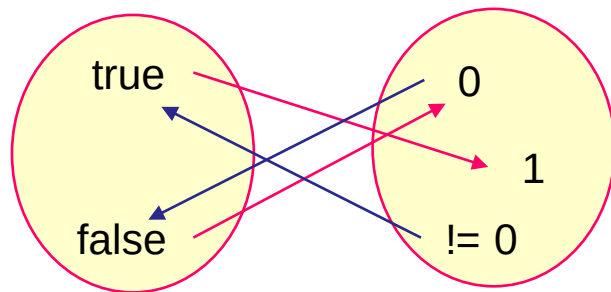
- Boolean type (*bool*)
- Character types (such as *char*): A single byte, capable of holding one character in the local character set.
- Integer types (such as *int*): An integer, typically reflecting the natural size of integers on a machine.
- Floating-point types (such as *double*): Floating point numbers.
- User-defined types: enumerations, void, pointers, arrays, references, data structures and classes

Booleans

- A boolean, **bool**, can have one of two values *true* or *false*.
- boolean $\leftrightarrow$ logical operations

```
bool accept()
{
    cout << "Do you want to proceed (y or n )? "; // write question
    char answer;
    cin >> answer; // read answer
    if (answer == 'y') return true;
    return false;
}
```

- boolean conversion:



boolean
values

integer
values

```
bool greater(int a, int b)
{
    return a > b;
}
```

```
void f(int a, int b)
{
    bool b1 = (a == b);
}
```

```
bool b = 7; // bool(7) is true, so b becomes true.
int i = true; // int(true) is 1, so i becomes 1.
```

- boolean literals: *true*, *false*.

Character types

- A variable of type `char` can hold a character of the implementation's **character set**.

```
char c = 'a';
```

- C++ character set consists of 96 characters:

a b c d e f g h i j k l m n o p q r s t u v w x y z

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

0 1 2 3 4 5 6 7 8 9

_ { } [] # () < > % : ; . ? * + - / ^ & | ~ ! = , \ " '

The space, Control characters: horizontal tab, vertical tab, form feed, and new-line.

- The size of character is implementation-defined. Almost universally, characters are 8 bits. So there can be 256 different characters
 - EBCDIC
 - ASCII

Character types cont.

- Each character constant has an integer value.

- The notation `int(c)` gives the integer value for a character `c`.

ASCII character set

			'A'	'B'		'a'	'b'	
0	1	...	65	66	...	97	...	255

```
// read a character from input and write its integer value
#include <iostream>
int main()
{
    char c;
    std::cin >> c;
    std::cout << "The value of '" << c << "' is " << int(c) << std::endl;

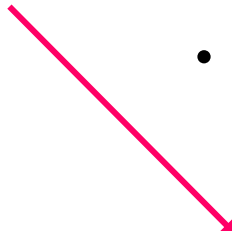
    return 0;
}
```

- A type `wchar_t` is provided to hold characters of a larger character set such as *Unicode*. The size of `wchar_t` is implementation-defined.
- Character type is integral type: Arithmetic operation: `+`, `-`, `++`, `--`, `*`, `/`, ...
- character literals: `'a'`, `'0'`, `'!'`, ... The type of character literals are `char`.
- wide character literals: `L'ab'`, `L'ac'`. The type of character literals are `wchar_t`.

Integer types

- Integer types:

- `short int` (short)



- `int`

- `long int` (long)

- The size of integer types are *implementation-defined*.

- Integer literals:

- *decimal*

- *octal*

- *hexadecimal*

- *character literals*

```
int i;  
short si = 5;  
long int li = 1000000000;
```

decimal:	0 2 6 3 8 3
octal:	00 02 077 0123
hexadecimal:	0x0 0x2 0x3f 0x53

- The letters a , b , c , d , e , and f , or their uppercase equivalents, are used to represent 10 , 11 , 12 , 13, 14, and 15, respectively.

- By default, a integer literal is of type *int*.

Floating-point types

- The floating point types represent floating point numbers.
- floating-point types:
 - `float` (single precision)
 - `double` (double precision)
 - `long double` (extended precision)
- The exact meaning of single, double, and extended precision is *implementation-defined*.
- By default, a floating-point literal is of type *double*.

```
float f; // uninitialized
double d2 = 1; // convert to 1.0
double pi = 3.14;
long double e = 2.7182818284590452354;
```

```
1.23    .23    1.    6.02e23
```

The sizes of fundamental types

- Sizes of C++ objects are expressed in terms of multiples of the size of a char.
- `sizeof` operator: size of types and size of objects

```
1  ≡ sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long)
    sizeof(char) <= sizeof(wchar_t)
    sizeof(float) <= sizeof(double) <= sizeof(long double)
```

- A `char` has at least 8 bits.
- A `short` has at least 16 bits.
- A `long` has at least 32 bits.

For most applications, one could simply use `bool` for logical values, `char` for characters, `int` for integer values, and `double` for floating-point values.

The sizes of fundamental types- an example

The sizes of fundamental types- an example

- Write a program that prints the sizes of the fundamental types and a few variables of fundamental types.

The sizes of fundamental types- an example

- Write a program that prints the sizes of the fundamental types and a few variables of fundamental types.

```
#include <iostream>

int main()
{
    // fundamental types
    std::cout << "On this machine ..." << '\n';
    std::cout << "Size of bool is " << sizeof(bool) << '\n';
    std::cout << "Size of char is " << sizeof(char) << '\n';
    std::cout << "Size of int is " << sizeof(int) << '\n';
    std::cout << "Size of long int is " << sizeof(long) << '\n';
    std::cout << "Size of double is " << sizeof(double) << '\n';
    // some objects
    int i;
    long double d;
    bool b = false;
    std::cout << "Size of i is " << sizeof(i) << '\n';
    std::cout << "Size of d is " << sizeof(d) << '\n';
    std::cout << "Size of b is " << sizeof(b) << '\n';

    return 0;
}
```

Digit separators



- In C++, the single-quote character may be used arbitrarily as a digit separator in numeric literals, both integer literals and floating-point literals.

```
long long wp = 7'000'000'000; // World population
int shares = 500'000'000; // # shares at TSE
double price = 400.'00; // the closing price of each share in IRR
```



Use digit separators to make large literals readable.

Rferences and further readings

- [KM00] Andrew Koenig and Barbara Moo. Accelerated C++: Practical Programming in C++, Addison Wesley, 2000.
- [Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.
- [Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.
- [Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

